



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA:3712

Software Testing

CO1-AT2- SHORT ANSWER TEST

Name: B S Reshma

Reg no. 192521342

QUESTION 1

A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.

QUESTION 2

A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.

QUESTION 3

During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.

QUESTION 4

A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.

QUESTION 5

A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.

QUESTION 6

During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement

QUESTION 7

A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.

QUESTION 8

A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.

QUESTION 9

A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.

QUESTION 10

An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.

QUESTION 11

A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.

QUESTION 12

A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.

QUESTION 13

A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.

QUESTION 14

During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.

QUESTION 15

A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.

QUESTION 16

A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.

QUESTION 17

A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.

QUESTION 18

An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.

QUESTION 19

A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.

QUESTION 20

A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.

QUESTION 21

A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.

QUESTION 22

A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.

QUESTION 23

A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.

QUESTION 24

A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.

QUESTION 25

A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.

QUESTION 26

A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.

QUESTION 27

A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.

QUESTION 28

A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.

QUESTION 29

A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.

QUESTION 30

A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.

ANSWERS

1.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Testing as an engineering activity is a planned, systematic and measurable process. Random testing should be replaced with requirements-based, risk-based and traceable test design.

PROCEDURE / STEPS FOR TESTING

1. Review requirements and identify business risks.
2. Identify positive, negative, boundary, exception and integration conditions.
3. Design traceable test cases with inputs, preconditions, steps and expected results.
4. Prioritize high-risk functions.
5. Execute tests and compare actual with expected results.
6. Log reproducible defects with evidence.
7. Retest fixes and run regression tests.
8. Review coverage and defect trends for improvement.

EXAMPLE

For a registration system, instead of checking a few random registrations, design cases for valid data, missing fields, duplicate email, invalid email, maximum field length and password rules.

RESULT

The redesigned approach provides repeatable execution, better coverage, earlier defect detection and measurable evidence of quality.

CONCLUSION

Testing should be treated as an engineering discipline: planned, traceable, risk-based, repeatable and measurable rather than random.

2.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

A defined testing process gives banking software a repeatable lifecycle covering requirements, test design, execution, defect management, retesting and regression.

PROCEDURE / STEPS FOR TESTING

- Review critical banking requirements.
- Identify transaction risks and priorities.

- Prepare unit, integration, system, security, performance and regression tests.
- Create positive, negative and boundary cases.
- Execute with controlled test data.
- Record, prioritize and track defects.
- Retest and regress after fixes.
- Apply release entry/exit criteria.

EXAMPLE

For an ATM, test valid withdrawal, insufficient balance, invalid PIN, daily-limit violation, zero amount and network interruption under a defined test cycle.

RESULT

The banking system gains consistent testing, fewer transaction failures and stronger release confidence.

CONCLUSION

A defined testing process is essential in banking because transaction defects can directly affect money, records and customer trust.

3.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Testing in the SDLC should begin during requirements and continue through design, coding, integration, system, acceptance and maintenance phases.

PROCEDURE / STEPS FOR TESTING

- Review requirements for ambiguity and testability.
- Review design and interfaces.
- Perform unit testing during coding.
- Perform incremental integration testing.
- Perform system and end-to-end testing.
- Support acceptance testing.
- Use regression testing for maintenance and changes.

EXAMPLE

For an e-commerce site, unit-test product logic, integration-test product-to-cart and payment interfaces, then system-test the complete search-to-checkout workflow.

RESULT

Testing becomes traceable across the SDLC and integration/system defects are detected earlier.

CONCLUSION

Process discipline across the SDLC makes quality a continuous activity instead of a final-stage activity.

4.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

An error is a human mistake, a defect is the resulting flaw in a work product, and a failure is the incorrect behavior observed when the software executes.

ALGORITHM / PROCEDURE

- Observe the mismatch.
- Identify the human mistake or incorrect decision.
- Locate the resulting defect in the requirement, design or code.
- Execute the relevant test and record the observed failure.
- Use expected and actual results to communicate the problem clearly.

EXAMPLE

If a valid login is expected to redirect to the dashboard but actually redirects elsewhere, the developer's mistake is an error, the incorrect code is the defect, and the observed incorrect behavior is the failure.

RESULT

The team can accurately communicate whether the issue is an error, defect or observed failure and select the appropriate corrective action.

CONCLUSION

Clear terminology improves communication between testers and developers and supports correct defect handling.

5.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

The principle of early testing states that testing activities should begin as early as possible so defects are detected close to where they are introduced.

PROCEDURE / STEPS FOR TESTING

- Review requirements early.
- Perform requirement and design reviews.
- Define acceptance criteria before implementation.
- Use unit and component testing during coding.

- Integrate incrementally.
- Track the phase of introduction and detection for defects.
- Retest and regress corrections.

EXAMPLE

If a password requirement says eight characters but is ambiguous, a requirement review can resolve it before different developers implement different interpretations.

RESULT

Defects are detected earlier, reducing rework, debugging effort and late-stage correction cost.

CONCLUSION

Early testing is an effective principle for reducing the cost and impact of defects.

6.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

A tester can improve quality during requirements analysis by checking ambiguity, completeness, consistency, testability and acceptance criteria.

PROCEDURE / STEPS FOR TESTING

- Read every requirement for ambiguity and missing rules.
- Identify conflicting or incomplete conditions.
- Ask how each requirement will be verified.
- Suggest measurable acceptance criteria.
- Derive positive, negative and boundary cases.
- Maintain requirement-to-test traceability.
- Review requirement changes for testing impact.

EXAMPLE

A healthcare requirement saying only authorized staff may view patient records should define roles, permissions, failed-login behavior and acceptance criteria before coding.

RESULT

Requirement defects are detected before implementation, reducing rework and improving testability.

CONCLUSION

Tester participation during requirements improves both software quality and the effectiveness of later testing.

7.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Unclear requirements can create defects because developers may interpret the same business rule differently. Clear, measurable requirements reduce this risk.

PROCEDURE / STEPS FOR TESTING

- Identify the unclear business rule.
- Clarify it with stakeholders.
- Define exact inputs, calculations, rounding and exceptions.
- Document acceptance criteria.
- Trace the requirement into design and code.
- Create representative and boundary tests.
- Regression-test future changes.

EXAMPLE

If overtime pay is not defined precisely, developers may choose different rates or rounding rules. Clear overtime rules can be converted directly into test cases.

RESULT

Clear requirements reduce interpretation differences and prevent many calculation defects from reaching later phases.

CONCLUSION

Requirement clarity is a fundamental form of defect prevention because precise requirements enable precise implementation and testing.

8.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

A crash at an input limit is a boundary-related functional defect. Boundary Value Analysis targets the minimum, maximum and neighboring values where defects commonly occur.

ALGORITHM / PROCEDURE

1. Identify the valid input range.
2. Test the minimum and maximum.
3. Test values immediately below and above each boundary.
4. Compare actual and expected behavior.
5. Record boundary failures.

6. Retest after correction and add the case to regression testing.

EXAMPLE

For an allowed age range of 18–60, test 17, 18, 19, 59, 60 and 61 to expose boundary handling problems.

RESULT

Boundary-focused testing exposes failures at limits before users encounter them.

CONCLUSION

Boundary Value Analysis should be included whenever input limits can affect system behavior.

9.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

A defect repository preserves information about previous failures, their causes and corrections. This information can be converted into stronger regression and preventive tests.

PROCEDURE / STEPS FOR TESTING

1. Collect previous defect reports.
2. Classify defects by module, cause, severity and environment.
3. Find recurring patterns.
4. Convert important historical defects into regression cases.
5. Add boundary and negative cases for recurring failures.
6. Run the relevant regression suite after changes.
7. Use trends for future test planning.

EXAMPLE

If a discount defect has appeared in several releases, add cases for multiple discounts, maximum discounts and invalid coupon combinations to regression testing.

RESULT

Historical defect knowledge is reused to strengthen test design and reduce repeated defects.

CONCLUSION

A defect repository is useful only when historical defects are actively reused for regression, prevention and test improvement.

10.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Incorrect ATM withdrawal behavior violates a specified transaction function, so it is a functional defect. Transaction rules must be tested with normal and exceptional conditions.

PROCEDURE / STEPS FOR TESTING

1. Identify the expected withdrawal rules.
2. Test valid amount, balance, PIN and limits.
3. Test insufficient balance and invalid input.
4. Verify cash dispensing and balance deduction.
5. Test timeout, cancellation and network failure.
6. Retest the correction and run regression tests.

EXAMPLE

If an ATM dispenses ₹2,000 but deducts ₹10,000, the withdrawal transaction function is incorrect and should fail a functional test.

RESULT

Systematic functional testing can detect incorrect transaction logic before deployment.

CONCLUSION

Critical transaction functions require systematic functional and exception testing before release.

11.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Defect clustering means that a relatively small number of modules often contain a large proportion of defects. Testing should therefore be risk-focused.

PROCEDURE / STEPS FOR TESTING

1. Collect defects by module.
2. Count severity and business impact.
3. Identify high-defect modules.
4. Analyze common causes.
5. Increase testing depth and review effort in those modules.
6. Reassess priorities using new release data.

EXAMPLE

If Module A contains most high-severity defects, increase review, integration, regression and exploratory testing effort there instead of distributing effort equally.

RESULT

Testing resources are concentrated where important defects are most likely to occur.

CONCLUSION

Defect clustering allows limited testing resources to be allocated according to actual risk.

12.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Slow behavior under heavy load is a performance or non-functional defect. It should be investigated using load, stress and performance measurements.

ALGORITHM / PROCEDURE

1. Measure response time and throughput.
2. Run load tests at expected usage.
3. Increase load to identify degradation.
4. Run stress tests beyond expected capacity.
5. Monitor CPU, memory and other resources.
6. Compare results with performance requirements.
7. Retest after optimization.

EXAMPLE

If the site works for 50 users but becomes very slow for 1,000 simultaneous users, perform load and stress testing and measure response time and resource usage.

RESULT

Correct classification directs the team toward performance testing and measurable resource analysis.

CONCLUSION

Performance defects require performance-oriented measurements and tests rather than only functional checks.

13.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Historical defect information is valuable regression input because previously failed conditions are proven risk areas and should remain covered after changes.

PROCEDURE / STEPS FOR TESTING

1. Collect historical defects.
2. Identify severe, frequent and change-related defects.
3. Convert them into permanent regression tests.
4. Prioritize cases based on current risk.

5. Execute after each relevant change.
6. Add newly escaped defects to the regression suite.
7. Review and maintain the suite.

EXAMPLE

If previous releases repeatedly had checkout tax and payment-retry defects, those exact conditions should remain in the regression suite after checkout changes.

RESULT

Historical failures become reusable regression knowledge, improving coverage and efficiency.

CONCLUSION

Historical defect data provides practical evidence for selecting high-value regression tests.

14.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Testers reduce integration problems by clarifying interfaces, data formats, expected behavior and error handling with developers and stakeholders.

PROCEDURE / STEPS FOR TESTING

1. Clarify interface contracts and data formats.
2. Review expected inputs, outputs and errors.
3. Design interface and data-mapping tests.
4. Use incremental integration.
5. Report reproducible failures with data and environment.
6. Retest fixes and perform related regression.

EXAMPLE

If one service sends an amount in rupees and another interprets it as paise, an interface/data-mapping test can expose the integration error early.

RESULT

Clear collaboration and interface testing reduce integration misunderstandings and make failures easier to isolate.

CONCLUSION

Testers act as quality contributors and collaborators throughout integration, not merely as final-stage defect finders.

15.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Defect prevention focuses on eliminating the causes of recurring defects rather than only finding and fixing individual occurrences.

PROCEDURE / STEPS FOR TESTING

1. Collect recurring defects.
2. Group similar failures.
3. Perform root-cause analysis.
4. Identify whether the cause is requirements, design, code, process or environment.
5. Change the process or standards causing the defect.
6. Add preventive and regression tests.
7. Monitor later releases for recurrence.

EXAMPLE

If date calculations repeatedly fail across time zones, standardize date handling, review the design and add multi-time-zone regression tests.

RESULT

Addressing root causes reduces recurrence rather than repeatedly fixing the same symptom.

CONCLUSION

Defect prevention improves long-term quality by reducing the causes of recurring failures.

16.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Mobile testing must be standardized while accounting for device, operating-system, screen, network, permission and interruption differences.

PROCEDURE / STEPS FOR TESTING

1. Define supported devices and OS versions.
2. Use a standard test-case format.
3. Test core functions.
4. Test permissions, orientation, interruptions and notifications.
5. Test different network conditions.
6. Perform compatibility, performance and regression testing.
7. Record device, OS and build information in defects.

EXAMPLE

A banking app should be tested for login and fund transfer across supported devices, OS versions, screen sizes, network conditions and interruptions.

RESULT

Standardized mobile testing improves reliability across supported devices and environments.

CONCLUSION

A standardized mobile strategy combines functional testing with compatibility and mobile-specific condition testing.

17.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Testing evaluates whether software behaves as expected; debugging investigates the cause of an observed problem and changes the software to correct it.

ALGORITHM / PROCEDURE

1. Execute tests and observe incorrect behavior.
2. Record the failure and evidence.
3. Developer investigates the root cause.
4. Developer corrects the code or configuration.
5. Tester retests the correction.
6. Run regression tests to check for side effects.

EXAMPLE

For an ATM that deducts the wrong amount, testing discovers and reports the failure; debugging investigates the calculation and changes the faulty code.

RESULT

Testing detects and communicates incorrect behavior, while debugging locates and removes its cause.

CONCLUSION

Testing and debugging have different responsibilities but work together to improve software quality.

18.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Cross-browser testing verifies important functions and presentation across supported browsers, versions and operating environments.

PROCEDURE / STEPS FOR TESTING

1. Define supported browser/OS combinations.
2. Create a browser compatibility matrix.
3. Run common functional workflows across browsers.
4. Check layout, forms, JavaScript and responsive behavior.
5. Record browser/version for failures.
6. Retest fixes across affected environments.

EXAMPLE

If checkout works in Chrome but the button fails in another supported browser, the browser/version and exact workflow should be recorded and reproduced in the compatibility matrix.

RESULT

A browser matrix and repeatable compatibility tests reduce customer-facing cross-browser failures.

CONCLUSION

Cross-browser testing should be planned using supported environments and repeatable workflows.

19.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Validation testing checks whether inputs and business rules are enforced correctly, including invalid, missing, boundary and unexpected conditions.

PROCEDURE / STEPS FOR TESTING

1. List important logistics inputs and rules.
2. Test valid, invalid, missing and boundary values.
3. Check relationships between fields.
4. Verify business rules and dates.
5. Check cross-module data consistency.
6. Record validation failures and add them to regression testing.

EXAMPLE

A logistics system should reject a delivery date earlier than the dispatch date and should also test missing, equal and invalid dates according to the business rule.

RESULT

Systematic validation prevents incorrect data and business-rule violations from propagating through the system.

CONCLUSION

Proper validation improves reliability by ensuring that inputs and business rules are consistently enforced.

20.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Systematic test design derives test conditions from requirements, risks, input domains, interfaces, boundaries, states and historical defects rather than random selection.

PROCEDURE / STEPS FOR TESTING

1. Study requirements and identify test conditions.
2. Partition inputs into meaningful classes.
3. Identify boundaries and exceptional states.
4. Consider interfaces and business rules.
5. Use historical defects to add high-value tests.
6. Document expected results before execution.
7. Prioritize by risk and execute systematically.

EXAMPLE

For login, systematic tests include valid credentials, wrong password, unknown username, empty fields, password boundaries, locked account and session timeout.

RESULT

Systematic test design improves coverage, repeatability and defect detection compared with random execution.

CONCLUSION

Systematic test design is a core part of testing as an engineering activity.

21.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Integration testing verifies that modules exchange and preserve data correctly. Data mapping, formats, synchronization and error handling are important causes to investigate.

PROCEDURE / STEPS FOR TESTING

1. Identify modules and integration points.
2. Review field mappings and formats.
3. Test data transfer between modules.
4. Check synchronization and update behavior.
5. Test missing, duplicate and stale data.
6. Test failure recovery.

7. Retest corrected interfaces and regress related functions.

EXAMPLE

If allergy data exists in the patient module but is missing in the pharmacy module, test the interface mapping, format and synchronization between the two modules.

RESULT

Integration testing protects data consistency by checking the complete flow between modules.

CONCLUSION

Integration testing is essential when modules share important healthcare data.

22.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Testers contribute throughout the SDLC, from requirement reviews and design analysis to integration, system, acceptance and maintenance testing.

PROCEDURE / STEPS FOR TESTING

1. Requirements: review clarity and acceptance criteria.
2. Design: review workflows, risks and interfaces.
3. Implementation: plan unit/component and integration tests.
4. Integration: verify data flow.
5. System: validate end-to-end behavior.
6. Acceptance: support business validation.
7. Maintenance: regression-test changes and production fixes.

EXAMPLE

For a payment feature, identify timeout, duplicate-payment and failed-payment cases during requirements and design so they are tested before release.

RESULT

Early tester involvement reduces ambiguity and provides earlier quality feedback throughout development.

CONCLUSION

Early tester involvement improves quality by influencing requirements and design before defects become expensive.

23.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Peak-usage failures occur when normal-use testing does not cover realistic load and capacity conditions. Coverage must include normal, peak and exceptional usage.

PROCEDURE / STEPS FOR TESTING

1. Identify normal, peak and exceptional usage.
2. Define capacity and performance requirements.
3. Create realistic load scenarios.
4. Perform stress testing beyond expected capacity.
5. Test critical transactions under load.
6. Monitor response time, throughput and resources.
7. Test recovery and preserve important failures as regression cases.

EXAMPLE

A ticket-booking system that works for 100 users may fail at 5,000 users; load and stress tests can expose database, session or server-capacity limits.

RESULT

Peak-load failures are identified before production, improving availability and capacity confidence.

CONCLUSION

Adequate coverage must include realistic peak and stress conditions, not only normal usage.

24.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

A defect repository needs standardized information such as ID, environment, severity, priority, reproduction steps, expected/actual results, evidence and status.

PROCEDURE / STEPS FOR TESTING

9. Create a unique defect ID.
10. Record module, build and environment.
11. Write clear reproduction steps.
12. Record expected and actual results.
13. Assign severity and priority.
14. Attach evidence where useful.
15. Track owner, status, fix and retest.
16. Link important defects to regression tests.

EXAMPLE

A login defect report should include build, browser, reproduction steps, expected result, actual result and evidence so the developer can reproduce it.

RESULT

Standardized defect documentation makes issues traceable from discovery through correction and retesting.

CONCLUSION

Good defect documentation supports communication, tracking, analysis and continuous improvement.

25.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Banking systems require risk-based and structured testing of critical transactions, including balances, transfers, limits, timeouts, retries and duplicate requests.

PROCEDURE / STEPS FOR TESTING

- Identify critical banking functions.
- Rank them by business risk.
- Design positive, negative, boundary and exception tests.
- Test transaction atomicity.
- Test timeout, retry, cancellation and duplicate requests.
- Perform integration, security, performance and regression testing.
- Use risk-based release criteria.

EXAMPLE

For fund transfer, test success, insufficient balance, invalid beneficiary, timeout, duplicate submission and network failure, including the rule that failed transfers must not incorrectly debit the sender.

RESULT

Risk-based structured testing improves reliability of critical banking transactions.

CONCLUSION

Structured risk-based testing is necessary for reliable banking transactions.

26.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Edge cases include minimum, maximum, empty, zero, negative, maximum-length and unusual combinations. Boundary testing deliberately targets these conditions.

PROCEDURE / STEPS FOR TESTING

- Identify input ranges.

- Test exact boundaries and neighboring values.
- Include empty, null, zero and negative values where applicable.
- Test maximum lengths and large values.
- Test unusual valid combinations.
- Test invalid formats.
- Add important edge cases to regression.

EXAMPLE

For marks from 0 to 100, test 0, 1, 99, 100, -1 and 101, plus empty or non-numeric values if applicable.

RESULT

Edge and boundary cases reveal failures that normal-value testing can miss.

CONCLUSION

Edge and boundary testing increases the chance of finding defects hidden outside normal operating conditions.

27.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Process-based testing standardizes lifecycle activities, templates, entry/exit criteria, defect rules, traceability, regression and metrics across modules.

PROCEDURE / STEPS FOR TESTING

- Define one common testing lifecycle.
- Use standard test-case and defect templates.
- Define entry and exit criteria.
- Use common severity and priority rules.
- Maintain traceability.
- Apply standard regression practices.
- Collect coverage and defect metrics.
- Review results and improve the process.

EXAMPLE

If one module receives detailed boundary testing while another receives only normal-value tests, a common test process can require both modules to meet the same minimum coverage categories.

RESULT

Standardization reduces process variation and ensures essential testing activities are not skipped.

CONCLUSION

A common testing process provides consistent quality across modules while still allowing risk-based additional testing.

28.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Robust input validation requires explicit rules for type, format, range and length and tests invalid, malformed, boundary and unexpected input conditions.

PROCEDURE / STEPS FOR TESTING

- Identify all user-controlled inputs.
- Define type, format, range and length rules.
- Test valid and invalid values.
- Test empty, null, special and malformed inputs where applicable.
- Verify validation at the appropriate application/server layer.
- Check that invalid input gives a controlled response rather than a crash.
- Add important cases to regression.

EXAMPLE

For a quantity field, test positive integers, zero, negative values, decimals, very large values, blanks and invalid text, ensuring invalid data is rejected safely.

RESULT

Strong input validation keeps the application predictable and robust when users provide unexpected data.

CONCLUSION

Input validation is essential for robust behavior under invalid and unexpected conditions.

29.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Defect leakage should be analyzed using production defect data and root-cause analysis. Escaped defects should become new regression and prevention inputs.

PROCEDURE / STEPS FOR TESTING

- Collect production defects.
- Classify them by type, severity and affected phase.
- Perform root-cause analysis.

- Identify missing requirements, coverage, environment or release controls.
- Add escaped defects to regression tests.
- Strengthen reviews and test data/environment where needed.
- Define release criteria for critical defects.
- Monitor leakage trends.

EXAMPLE

If production reveals a defect only when three discount rules are combined, add that combination to system and regression testing and review similar rule combinations.

RESULT

Production defects become learning inputs for improved regression coverage and prevention.

CONCLUSION

Reducing defect leakage requires learning from production failures and strengthening the testing process.

30.

AIM

To apply the appropriate software testing concept to the given situation and improve software quality, defect detection and reliability.

INTRODUCTION

Early defect detection combines requirement reviews, design reviews, unit/component testing, incremental integration, risk-based system testing and continuous feedback.

PROCEDURE / STEPS FOR TESTING

- Review requirements for ambiguity and inconsistency.
- Perform design reviews.
- Identify risks and critical functions early.
- Design tests alongside development.
- Use unit/component testing during implementation.
- Integrate incrementally.
- Perform risk-based system and acceptance testing.
- Track defect introduction and detection phases.
- Use regression and lessons learned for continuous improvement.

EXAMPLE

If a payment timeout requirement is unclear, a requirements review can define expected behavior for timeout, retry and duplicate-payment conditions before implementation.

RESULT

Defects are found closer to their source, reducing total rework, effort and cost.

CONCLUSION

Early defect detection is most effective when reviews, testing, metrics and feedback are integrated throughout the lifecycle.

Code of Conduct:

I B S Reshma (192521342) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

B S RESHMA

Rubrics for calculation:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand